

C# 프로그래밍

강의 13 — 종합 연습문제 100제
문제 + 정답 + 해설

동명대학교 게임학부

강영민

2026년 1학기

이 문제집의 목표

본 문제집은 한 학기 동안 다룬 C# 전체 내용을 **100문제**로 종합 점검한다. 단순 암기가 아닌 **왜 그렇게 동작하는가**를 설명할 수 있는지를 확인하는 데 중점을 둔다. 문제는 1부에서, 정답과 해설은 2부에서 한 번에 통합 제공한다.

Contents

제1부 문제집	3
1 기본 문법과 타입 시스템	3
2 조건문과 반복문	5
3 배열과 컬렉션	7
4 메서드와 매개변수	9
5 객체지향 프로그래밍	11
6 인터페이스와 추상화	13
7 예외 처리	15
8 제네릭	16
9 LINQ	18
10 비동기 프로그래밍	19
11 메모리와 런타임	21
12 고급 / 함수형 / 위임	22
13 설계 및 종합 사고	23
제2부 정답 및 해설	25
14 기본 문법과 타입 시스템 — 해설	25
15 조건문과 반복문 — 해설	27
16 배열과 컬렉션 — 해설	29
17 메서드와 매개변수 — 해설	31
18 객체지향 프로그래밍 — 해설	33
19 인터페이스와 추상화 — 해설	35
20 예외 처리 — 해설	37
21 제네릭 — 해설	38
22 LINQ — 해설	40
23 비동기 프로그래밍 — 해설	42
24 메모리와 런타임 — 해설	43
25 고급 / 함수형 / 위임 — 해설	45

26 설계 및 종합 사고 — 해설

46

제1부 문제집

안내

- 문제는 13개 부(部)로 나뉘어 있으며 모두 100문제이다.
- 코드 예시의 출력은 .NET 6 이상을 기준으로 한다.
- 정답과 해설은 본 문서의 제2부에 같은 번호로 정리되어 있다.

1. 기본 문법과 타입 시스템

문제 1. 다음 코드의 출력 결과를 예측하라.

```
int a = 5;
int b = a;
b += 3;
Console.WriteLine(a);
Console.WriteLine(b);
```

1. 출력 결과를 쓰시오.
2. 왜 이런 결과가 나오는지 설명하시오.
3. value type의 특징과 연결지어 설명하시오.

문제 2. 다음 코드에서 오류가 발생하는 이유를 설명하라.

```
double x = 3.14;
int y = x;
```

1. 컴파일 오류 원인
2. implicit conversion과 explicit conversion의 차이
3. 올바른 수정 방법 2가지

문제 3. 다음 코드의 결과를 설명하라.

```
object obj = 10;
Console.WriteLine(obj.GetType());
int n = (int)obj;
Console.WriteLine(n + 5);
```

1. boxing이 발생하는 위치
2. unboxing이 발생하는 위치
3. boxing/unboxing의 비용 문제

문제 4. 다음 두 선언의 차이를 설명하라.

```
var x = 10;
dynamic y = 10;
```

1. var는 언제 타입이 결정되는가?

2. `dynamic`은 언제 타입이 결정되는가?
3. 성능과 안정성 측면에서 비교하라.

문제 5. 다음 코드의 출력 결과는?

```
int max = int.MaxValue;
int x = max + 1;
Console.WriteLine(x);
```

1. 출력값을 예측하라.
2. 정수 overflow가 발생할 때의 기본 동작
3. `checked` / `unchecked` 키워드의 역할

문제 6. 다음 코드의 결과를 설명하라.

```
string s = "Hello";
s += " World";
Console.WriteLine(s);
```

1. `s += "World"`는 기존 문자열을 수정한 것인가?
2. string immutability의 의미
3. 반복적인 문자열 결합 시 `StringBuilder`를 쓰는 이유

문제 7. 다음 코드의 출력 결과를 예측하라.

```
int? a = null;
int? b = 10;
Console.WriteLine(a.HasValue);
Console.WriteLine(b ?? -1);
Console.WriteLine(a ?? -1);
```

1. nullable value type(`int?`)의 의미
2. `??` (null-coalescing) 연산자의 동작
3. `Nullable<T>` 구조체와의 관계

문제 8. `char`와 `string`의 차이를 설명하라.

1. 메모리 크기
2. value type / reference type 분류
3. 다음 코드의 결과는?

```
char c = 'A';
string s = "A";
Console.WriteLine(c == s[0]);
```

문제 9. `const`와 `readonly`의 차이를 설명하라.

1. 값이 결정되는 시점
2. 인스턴스마다 다른 값을 가질 수 있는가?

3. 다음 중 어느 것이 가능한가?

```
class Foo
{
    const int A = DateTime.Now.Year;    // (a)
    readonly int B = DateTime.Now.Year; // (b)
}
```

문제 10. nameof 연산자의 역할을 설명하라.

```
string field = "name";
string n1 = nameof(field);
Console.WriteLine(n1);
```

1. 출력 결과
2. 단순 "field" 문자열보다 좋은 이유
3. 예외 메시지 작성과 리팩토링에 어떻게 도움이 되는가?

2. 조건문과 반복문

문제 11. 다음 코드의 출력값은?

```
for (int i = 0; i < 5; i++)
{
    if (i == 2) continue;
    Console.Write(i + " ");
}
```

1. 출력 결과
2. continue의 역할
3. break와의 차이

문제 12. 다음 while문의 문제점을 설명하라.

```
int n = 1;
while (n < 100)
{
    Console.WriteLine(n);
}
```

1. 어떤 문제가 발생하는가?
2. 왜 발생하는가?
3. 수정 방법을 제시하라.

문제 13. 다음 switch문의 동작을 설명하라.

```
int x = 2;
switch (x)
{
    case 1:
        Console.WriteLine("one");
        break;
    case 2:
```

```

        Console.WriteLine("two");
        goto case 1;
    case 3:
        Console.WriteLine("three");
        break;
}

```

1. 출력 결과
2. C# switch의 fall-through 제한
3. C와의 차이

문제 14. 다음 switch expression(C# 8.0+) 의미를 설명하라.

```

string Describe(int n) => n switch
{
    < 0 => "negative",
    0 => "zero",
    > 0 and < 10 => "small positive",
    _ => "large positive"
};

```

1. _의 역할
2. pattern matching의 강점
3. 전통적 switch와 비교

문제 15. for, foreach, while의 차이를 설명하라.

1. 어떤 상황에서 for가 적절한가?
2. foreach가 요구하는 인터페이스
3. while이 가장 자연스러운 경우

문제 16. do-while과 while의 차이를 설명하라.

다음 중 어떤 경우에 do-while이 더 적절한가?

- 사용자 입력을 받아 검증 후 다시 받기
- 컬렉션 순회
- 조건 미리 체크가 필요한 경우

문제 17. 다음 코드처럼 중첩 루프에서 break만으로 안쪽 루프만 빠져나갈 수 있는 이유를 설명하라.

```

for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        if (j == 1) break;
        Console.WriteLine($"{i},{j}");
    }
}

```

1. 출력 결과
2. 바깥 루프까지 한 번에 빠져나가려면 어떤 방법이 있는가?

3. goto 사용은 권장되는가?

문제 18. 다음 코드의 의미를 설명하라.

```
object x = 42;
switch (x)
{
    case int n when n > 10:
        Console.WriteLine("big int");
        break;
    case string s:
        Console.WriteLine($"string {s}");
        break;
    default:
        Console.WriteLine("other");
        break;
}
```

1. type pattern의 의미
2. when 절의 역할
3. 어떤 케이스에 진입하는가?

3. 배열과 컬렉션

문제 19. 배열과 List<T>의 차이를 설명하라.

- 크기 변경 가능 여부
- 메모리 특성
- 성능 차이
- 사용 사례

문제 20. 다음 코드의 출력 결과는?

```
int[] arr = {1, 2, 3};
foreach (var x in arr)
{
    Console.Write(x * 2 + " ");
}
```

추가로, foreach 내부에서 원소 자체를 변경(x = 0)할 수 있는지 설명하라.

문제 21. 다음 두 선언의 차이를 설명하라.

```
int[,] a = new int[3, 4]; // 다차원 배열 (rectangular)
int[][] b = new int[3][]; // 가변 배열 (jagged)
```

1. 메모리 배치 차이
2. 각 행의 크기를 다르게 할 수 있는가?
3. 어떤 경우에 어떤 것을 쓰는가?

문제 22. Dictionary<TKey, TValue>의 동작을 설명하라.


```
var d = new Dictionary<string, int>();
d["A"] = 1;
d["A"] = 2;
Console.WriteLine(d["A"]);
Console.WriteLine(d["B"]);
```

1. 같은 키에 값을 또 넣으면?
2. 존재하지 않는 키 조회 시 동작은?
3. TryGetValue를 쓰는 이유

문제 23. HashSet<T>의 특성을 설명하라.

```
var set = new HashSet<int> { 1, 2, 2, 3 };
Console.WriteLine(set.Count);
```

1. 출력 결과
2. 내부 동작(해시 기반)
3. Contains의 평균 시간복잡도

문제 24. Queue<T>와 Stack<T>의 차이를 설명하라.
다음 코드의 결과는?

```
var q = new Queue<int>();
q.Enqueue(1); q.Enqueue(2); q.Enqueue(3);
Console.Write(q.Dequeue() + " ");

var s = new Stack<int>();
s.Push(1); s.Push(2); s.Push(3);
Console.Write(s.Pop());
```

문제 25. Array.Sort와 List<T>.Sort의 공통점과 차이를 설명하라.

1. 정렬 알고리즘
2. 안정 정렬인가?
3. 사용자 정의 비교자(Comparer<T>) 사용

문제 26. LinkedList<T>와 List<T>의 차이를 설명하라.

1. 내부 구조
2. 임의 위치 삽입/삭제의 시간복잡도
3. 인덱스 접근 비용
4. 어떤 경우에 LinkedList<T>를 선택하는가?

4. 메서드와 매개변수

문제 27. 다음 코드의 출력 결과를 설명하라.

```
void Change(int x) { x = 100; }  
int a = 5;  
Change(a);  
Console.WriteLine(a);
```

1. 결과 예측
2. 왜 값이 바뀌지 않는가?
3. call by value 설명

문제 28. ref, out, in 키워드의 차이를 설명하라.
각각에 대해:

- 전달 방식
- 호출 전 초기화 필요 여부
- 사용 목적
- 예제 코드

문제 29. params 키워드의 역할을 설명하라.

```
int Sum(params int[] nums)  
{  
    int total = 0;  
    foreach (var n in nums) total += n;  
    return total;  
}  
  
Console.WriteLine(Sum(1, 2, 3));  
Console.WriteLine(Sum(new[] {1, 2, 3}));
```

1. params가 없으면 호출 방식이 어떻게 달라지는가?
2. 마지막 매개변수에만 올 수 있는 이유

문제 30. 선택적 매개변수(default value)와 named arguments를 설명하라.

```
void Greet(string name, string prefix = "Hello", string suffix = "!")  
{  
    Console.WriteLine($"{prefix}, {name}{suffix}");  
}  
  
Greet("Alice");  
Greet("Bob", suffix: "?");
```

1. 출력 결과
2. named argument의 장점
3. 메서드 오버로딩과 비교했을 때의 이점

문제 31. 다음 두 메서드 중 컴파일 오류가 나는 것은?

```
void Foo(int x) { }
void Foo(double x) { }
void Foo(int x, int y = 0) { }
void Foo(int x) { return; } // (?)
```

메서드 오버로딩의 해석 규칙을 설명하라.

- 어떤 시그니처가 충돌하는가?
- 반환 타입은 오버로딩 기준에 포함되는가?

문제 32. extension method를 설명하라.

```
static class StringExt
{
    public static bool IsEmpty(this string s) => string.IsNullOrEmpty(s);
}

Console.WriteLine("").IsEmpty();
```

- 정의 규칙(static class, this)
- 실제로는 어떻게 컴파일되는가?
- 어떤 경우에 유용한가?

문제 33. local function의 의미를 설명하라.

```
int Outer(int x)
{
    int Square(int n) => n * n;
    return Square(x) + 1;
}
```

- 람다 식과의 차이
- 변수 캡처 동작
- 어떤 경우에 쓰는가?

문제 34. expression-bodied member의 의미를 설명하라.

```
class Point
{
    public int X { get; }
    public int Y { get; }
    public Point(int x, int y) => (X, Y) = (x, y);
    public double Length => Math.Sqrt(X * X + Y * Y);
    public override string ToString() => $"({X}, {Y})";
}
```

- => 의 의미
- 어떤 멤버에 쓸 수 있는가?
- 가독성 측면의 효과

5. 객체지향 프로그래밍

문제 35. 클래스와 구조체(struct)의 차이를 설명하라.

- value/reference type
- stack/heap (단순화한 일반론)
- 복사 방식
- 사용 시점

문제 36. 다음 코드의 출력 결과는?

```
class Person { public string Name; }

Person p1 = new Person();
p1.Name = "Alice";

Person p2 = p1;
p2.Name = "Bob";

Console.WriteLine(p1.Name);
```

1. 결과 예측
2. reference type 관점에서 설명
3. shallow copy 개념 설명

문제 37. 상속(inheritance)의 장점과 단점을 설명하라.
다음 용어를 정의하라.

- base class
- derived class
- override
- virtual
- polymorphism

문제 38. 접근 제한자 public, private, protected, internal, protected internal, private protected를 정리하라.

각각:

- 어디서 접근 가능한가?
- 같은 어셈블리/다른 어셈블리에서의 차이

문제 39. 다음 코드를 보고 생성자 체이닝(constructor chaining)을 설명하라.

```
class Animal
{
    public string Name;
    public Animal() : this("Unknown") { }
    public Animal(string name) { Name = name; }
}

class Dog : Animal
{
    public Dog(string name) : base(name) { }
}
```

1. `this(...)`의 의미
2. `base(...)`의 의미
3. 호출 순서

문제 40. `static` 클래스와 `static` 멤버를 설명하라.

1. `static` 클래스가 가질 수 없는 것
2. `static` 생성자의 호출 시점
3. 일반 클래스와 메모리 차이

문제 41. `partial` 클래스의 용도를 설명하라.

```
// File1.cs
partial class Form { void HandleClick() { } }

// File2.cs
partial class Form { void HandleHover() { } }
```

1. 어떤 도구가 이를 활용하는가? (예: Windows Forms, EF)
2. 컴파일 시 처리 방식

문제 42. `sealed` 클래스의 의미와 사용 이유를 설명하라.

1. 상속을 막는다는 게 무엇을 의미하는가?
2. 성능 측면의 효과(가상 호출 최적화)
3. `sealed override`는 무엇인가?

문제 43. 다음 코드의 의미를 설명하라.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; private set; }
    public string FullName => $"{Name} ({Age})";
}
```

1. `auto-property`
2. `private set`의 의미
3. `expression-bodied property`의 효과

문제 44. `nested class`(중첩 클래스)의 의미와 사용 시점을 설명하라.

```
class Outer
{
    private int data;
    public class Inner
    {
        // 어떤 멤버에 접근 가능한가?
    }
}
```

1. 중첩 클래스는 외부 클래스 인스턴스에 자동으로 접근 가능한가?
2. 자바의 inner class와 차이
3. 사용 시점

문제 45. 객체 초기화자(object initializer)의 동작을 설명하라.

```
var p = new Person { Name = "Alice", Age = 30 };
```

1. 생성자 호출 후 어떤 순서로 동작하는가?
2. 어떤 멤버를 초기화할 수 있는가?
3. 가독성에 어떤 영향을 미치는가?

문제 46. record 타입(C# 9.0+)의 특성을 설명하라.

```
record Point(int X, int Y);

var p1 = new Point(1, 2);
var p2 = new Point(1, 2);
Console.WriteLine(p1 == p2);
```

1. 일반 클래스와의 핵심 차이
2. value-based equality
3. with 식의 의미
4. record struct와의 차이

6. 인터페이스와 추상화

문제 47. abstract class와 interface의 차이를 설명하라.

- 다중 상속 가능 여부
- 구현 포함 가능 여부
- 사용 목적
- 언제 interface를 쓰는가

문제 48. 다음 코드에서 다형성이 어떻게 동작하는지 설명하라.

```
class Animal
{
    public virtual void Speak() => Console.WriteLine("Animal");
}

class Dog : Animal
{
    public override void Speak() => Console.WriteLine("Woof");
}

Animal a = new Dog();
a.Speak();
```

1. 출력 결과
2. 정적 타입과 동적 타입
3. virtual table(vtable) 개념

문제 49. C# 8.0의 default interface method를 설명하라.

```
interface ILogger
{
    void Log(string msg);
    void LogError(string msg) => Log("[ERROR] " + msg);
}
```

1. 기존 인터페이스 규칙과 어떻게 다른가?
2. 다이아몬드 상속 문제는 어떻게 처리되는가?
3. 언제 유용한가?

문제 50. 명시적 인터페이스 구현(explicit interface implementation)을 설명하라.

```
interface IA { void Show(); }
interface IB { void Show(); }

class Foo : IA, IB
{
    void IA.Show() => Console.WriteLine("IA");
    void IB.Show() => Console.WriteLine("IB");
}
```

1. 동일 이름 메서드 충돌을 어떻게 해결하는가?
2. 호출 방법
3. 가시성과 접근 제한자

문제 51. IEnumerable<T>와 IEnumerator<T>를 설명하라.

1. foreach가 동작하기 위해 필요한 인터페이스는?
2. MoveNext/Current의 역할
3. yield return이 자동 생성하는 것

문제 52. IDisposable과 using 문을 설명하라.

```
using (var stream = new FileStream("a.txt", FileMode.Open))
{
    // ...
}
```

1. using이 컴파일 시 어떻게 변환되는가?
2. unmanaged resource 해제의 의미
3. using 선언문(C# 8.0)의 차이

문제 53. IComparable<T>와 IComparer<T>의 차이를 설명하라.

1. “스스로 비교”와 “외부에서 비교”의 차이
2. `Array.Sort` 가 둘 중 무엇을 요구하는가?
3. 정렬에서 안정성과의 관계

문제 54. 인터페이스 분리 원칙(Interface Segregation Principle)을 설명하라.
다음 인터페이스의 문제점을 지적하라.

```
interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
}
```

1. 로봇 작업자에게 `Eat`이 필요한가?
2. 어떻게 분리해야 하는가?

7. 예외 처리

문제 55. 다음 코드의 문제를 설명하라.

```
int x = 0;
int y = 10 / x;
```

1. 어떤 예외가 발생하는가?
2. try-catch로 처리하라.
3. finally의 역할 설명

문제 56. 예외(Exception)를 “남용”하면 안 되는 이유를 설명하라.

- 성능
- 코드 가독성
- 논리 오류 은폐

문제 57. `throw`와 `throw ex`의 차이를 설명하라.

```
try { /* ... */ }
catch (Exception ex)
{
    throw;    // (a)
    throw ex; // (b)
}
```

1. 두 경우 stack trace의 차이
2. 어느 것이 바람직한가?
3. 새 예외로 감쌀 때(`throw new XxxException("...", ex)`)는 언제 쓰는가?

문제 58. 커스텀 예외 클래스를 만드는 방법을 설명하라.


```
class InvalidScoreException : Exception
{
    public int Score { get; }
    public InvalidScoreException(int score, string msg) : base(msg)
    {
        Score = score;
    }
}
```

- 어떤 클래스를 상속해야 하는가?
- 권장 생성자 패턴
- 어떤 정보를 추가로 담을 수 있는가?

문제 59. exception filter(when 절)의 의미를 설명하라.

```
try { /* ... */ }
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound)
{
    // ...
}
```

- 일반 if 분기와 차이
- stack 보존 측면의 장점
- 사용 시점

문제 60. try-finally 패턴이 필요한 이유를 설명하라.

```
var conn = OpenConnection();
try
{
    DoWork(conn);
}
finally
{
    conn.Close();
}
```

- 예외 발생 시 Close가 호출되는가?
- using으로 대체 가능한 조건은?
- 자원 누수의 의미

8. 제네릭

문제 61. 제네릭(Generic)의 장점을 설명하라.

List<int>가 ArrayList보다 좋은 이유:

- 타입 안전성
- 박싱/언박싱
- 코드 재사용성

문제 62. 다음 제약(constraint)들의 의미를 설명하라.

```
class Foo<T>
    where T : class, IComparable<T>, new()
{
}
```

각 제약을 정리하라:

- class
- struct
- new()
- interface
- base class constraint
- notnull, unmanaged

문제 63. 공변(out)과 반공변(in)을 설명하라.

```
IEnumerable<string> strings = new List<string>();
IEnumerable<object> objects = strings; // OK
```

1. 왜 IEnumerable<T>는 공변인가?
2. Action<T>는 반공변인 이유
3. 배열 공변의 위험성

문제 64. 제네릭 메서드와 제네릭 클래스의 차이를 설명하라.

```
class Box<T> { public T Value; }
static T First<T>(IEnumerable<T> xs) => xs.GetEnumerator().Current;
```

1. 타입 추론(type inference)이 가능한 쪽
2. 인스턴스화 시점
3. 어떤 상황에서 메서드만 제네릭으로 해야 하는가?

문제 65. 컬렉션 초기화자(collection initializer)의 동작을 설명하라.

```
var list = new List<int> { 1, 2, 3 };
var dict = new Dictionary<string, int> { ["A"] = 1, ["B"] = 2 };
```

1. 어떤 메서드/인터페이스가 필요한가?(Add, IEnumerable)
2. dictionary initializer의 두 가지 문법
3. 사용자 정의 컬렉션에 적용하는 방법

문제 66. 다음 동시성 컬렉션의 용도를 설명하라.

- ConcurrentDictionary<TKey, TValue>
- ConcurrentQueue<T>
- BlockingCollection<T>

1. 일반 Dictionary가 멀티스레드에서 위험한 이유
2. lock 기반 동기화와의 비교
3. 어떤 시나리오에 어떤 컬렉션이 적합한가?

9. LINQ

문제 67. 다음 LINQ 코드의 의미를 설명하라.

```
var result = numbers
    .Where(x => x % 2 == 0)
    .Select(x => x * 10);
```

1. Where의 역할
2. Select의 역할
3. deferred execution 설명

문제 68. LINQ의 장점과 단점을 설명하라.

- 가독성
- 함수형 스타일
- 성능 문제
- 디버깅 난이도

문제 69. deferred execution과 immediate execution의 차이를 설명하라.

```
var q = numbers.Where(x => x > 0);
numbers.Add(100);
foreach (var x in q) Console.Write(x + " ");
```

1. q가 평가되는 시점
2. ToList()가 동작을 어떻게 바꾸는가?
3. 부작용(side effect) 측면의 위험

문제 70. GroupBy와 Aggregate의 차이와 사용법을 설명하라.

```
var byLength = words.GroupBy(w => w.Length);
var sum = numbers.Aggregate(0, (acc, x) => acc + x);
```

1. GroupBy 결과의 타입
2. Aggregate의 일반화된 의미
3. Sum/Max/Average와의 관계

문제 71. LINQ의 Join 연산을 설명하라.

```
var q = from c in customers
        join o in orders on c.Id equals o.CustomerId
        select new { c.Name, o.Amount };
```

1. SQL JOIN과의 유사점/차이점
2. inner join과 left join을 LINQ에서 어떻게 표현하는가?
3. GroupJoin의 의미

문제 72. `ToList()`, `ToArray()`, `ToDictionary()`, `ToHashSet()`의 의미를 설명하라.

1. 모두 immediate execution인 이유
2. 메모리 측면의 비용
3. 사용 시 주의점

문제 73. method syntax와 query syntax의 차이를 설명하라.

```
// method syntax
var a = items.Where(x => x > 0).Select(x => x * 2);

// query syntax
var b = from x in items where x > 0 select x * 2;
```

1. 컴파일 후 결과는 같은가?
2. 표현력 측면의 차이
3. 어느 쪽이 더 일반적인가?

문제 74. `IEnumerable<T>`와 `IQueryable<T>`의 차이를 설명하라.

1. 실행 위치(메모리 vs DB)
2. expression tree의 역할
3. EF/LINQ to SQL과 LINQ to Objects의 차이

10. 비동기 프로그래밍

문제 75. `async`와 `await`의 역할을 설명하라.

- 왜 필요한가?
- thread와의 관계
- UI freeze 문제

문제 76. 다음 코드의 실행 흐름을 설명하라.

```
async Task Test()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}
```

1. `await` 직전과 직후의 thread는 같을 수 있는가?
2. `Test()` 호출이 즉시 반환되는가?
3. state machine 변환

문제 77. `Task`와 `Thread`의 차이를 설명하라.

1. 추상화 수준
2. 스레드 풀과의 관계
3. async 메서드는 새 스레드를 만드는가?

문제 78. Task.WhenAll과 Task.WhenAny의 차이를 설명하라.

```
var t1 = LoadAsync(1);
var t2 = LoadAsync(2);
await Task.WhenAll(t1, t2);
```

1. 동시 실행 시 성능 효과
2. 예외 처리의 차이(AggregateException)
3. WhenAny의 사용 시나리오(타임아웃 등)

문제 79. ConfigureAwait(false)의 의미를 설명하라.

```
await stream.ReadAsync(buf).ConfigureAwait(false);
```

1. SynchronizationContext와의 관계
2. 라이브러리 코드에서 자주 쓰이는 이유
3. UI 코드에서 주의점

문제 80. async void의 위험성을 설명하라.

1. 예외 전파의 차이
2. 호출자가 완료를 기다릴 수 없음
3. 어떤 경우에만 허용되는가?(이벤트 핸들러)

문제 81. CancellationToken의 동작을 설명하라.

```
async Task DoWork(CancellationToken token)
{
    while (!token.IsCancellationRequested)
    {
        await Task.Delay(100, token);
    }
}
```

1. 협력적 취소(cooperative cancellation)의 의미
2. OperationCanceledException
3. CancellationTokenSource와의 관계

문제 82. SynchronizationContext의 역할을 설명하라.

1. WinForms/WPF에서의 UI 스레드 복귀 메커니즘
2. ASP.NET Core가 SynchronizationContext를 갖지 않는 이유
3. 비동기 코드 이식 시 주의점

11. 메모리와 런타임

문제 83. Garbage Collector(GC)의 역할을 설명하라.

- managed memory
- heap
- reference tracking
- stop-the-world

문제 84. stack과 heap의 차이를 설명하라.

비교 항목:

- 저장 대상
- 속도
- lifetime
- allocation 방식

문제 85. 세대별 GC(Gen 0, Gen 1, Gen 2)를 설명하라.

1. 왜 세대를 나누는가?
2. 단명/장수 객체 가설
3. Gen 0 수집이 가장 빠른 이유

문제 86. IDisposable과 finalizer(~ClassName())의 차이를 설명하라.

1. 호출 시점이 결정적/비결정적
2. 두 가지를 모두 구현해야 하는 패턴(Disposable Pattern)
3. unmanaged resource를 다룰 때의 의미

문제 87. Large Object Heap(LOH)을 설명하라.

1. 임계 크기(약 85,000 바이트) 의미
2. 일반 heap과의 차이
3. 큰 배열 할당 시 주의점

문제 88. Span<T>와 Memory<T>의 의미를 설명하라.

1. 왜 도입되었는가?(zero-copy slice)
2. heap이 아닌 stack 기반 참조
3. async 메서드에서 Span<T> 사용 제약과 Memory<T>의 역할

12. 고급 / 함수형 / 위임

문제 89. 다음 코드의 문제를 설명하라.

```
List<int> list = null;
Console.WriteLine(list.Count);
```

- 어떤 예외가 발생하는가?
- nullable reference type 관점 설명
- 안전한 코드로 수정 (?, ??, null 체크)

문제 90. delegate의 개념을 설명하라.

- 함수 포인터와의 관계
- event와 연결
- 람다식과 관계

문제 91. 다음 코드의 의미를 설명하라.

```
Func<int, int> square = x => x * x;
Console.WriteLine(square(5));
```

- Func<int, int>의 의미
- 람다식 구조 `x => x * x`
- delegate와 어떻게 연결되는가?

문제 92. 다음 코드에서 closure(변수 캡처)가 어떻게 동작하는지 설명하라.

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
{
    actions.Add(() => Console.Write(i + " "));
}
foreach (var a in actions) a();
```

- 출력 결과 (C# 5.0 이전 / 이후 동작 차이도 함께)
- foreach 변수 캡처는 어떻게 다른가?
- 의도한 결과를 얻으려면 어떻게 수정?

문제 93. event와 delegate의 차이를 설명하라.

```
public class Button
{
    public event EventHandler Clicked;
    public void Raise() => Clicked?.Invoke(this, EventArgs.Empty);
}
```

- event는 외부에서 Invoke 가능한가?
- event가 제공하는 캡슐화

3. +=, -= 만 허용되는 이유

문제 94. anonymous type을 설명하라.

```
var p = new { Name = "Alice", Age = 30 };
Console.WriteLine(p.Name);
```

1. 컴파일러가 만드는 타입
2. 불변(immutable) 여부
3. LINQ projection에서 자주 쓰이는 이유

13. 설계 및 종합 사고

문제 95. 다음 중 struct로 만드는 것이 더 적절한 것은 무엇인가? 각각 이유와 함께 답하라.

- 좌표(Point)
- 은행 계좌(Account)
- 게임 캐릭터(Player)
- 색상(Color)
- 사람(Person)

문제 96. “상속보다 합성(composition)을 선호하라”는 말의 의미를 설명하라.
다음 시나리오로 답하라:

- class Duck : Bird { ... } 식 상속 설계의 문제점
- “행동(behavior)”을 인터페이스/필드로 주입하는 방식의 이점
- 예시 코드를 직접 작성하라

문제 97. C#이 *managed language*라는 의미를 설명하라.

- CLR
- GC
- 메모리 안전성
- unmanaged language(C/C++)와의 차이

문제 98. 다음 개념들을 하나의 프로그램 흐름 안에서 연결하여 설명하라.

- 클래스
- 인터페이스
- 제네릭
- LINQ
- async/await

- 예외 처리

“실제 응용 프로그램(예: 웹 API 클라이언트, 데이터 로더 등) 관점”에서 서술하라.

문제 99. SOLID 원칙 5가지를 C# 관점에서 설명하라.

- S: Single Responsibility
- O: Open/Closed
- L: Liskov Substitution
- I: Interface Segregation
- D: Dependency Inversion

각각에 대해 짧은 C# 코드 예시(또는 위반 예시)를 보여라.

문제 100. C# 언어와 .NET 플랫폼의 관계를 설명하라.

다음은 포함하라:

- CLR(Common Language Runtime)
- CTS(Common Type System)
- CLS(Common Language Specification)
- IL(Intermediate Language)과 JIT
- .NET Framework vs .NET Core vs .NET 5+(통합 .NET)
- C#이 다른 .NET 언어(F#, VB.NET)와 어떻게 상호운용되는가?

제2부 정답 및 해설

안내

번호는 제1부의 문제 번호와 일치한다. 코드 예시 출력은 .NET 6+ 기준이며, 초보자가 “왜 그렇게 동작하는가”를 이해할 수 있도록 동작 원리(value/reference, boxing, vtable, GC 세대, 상태 머신 등)까지 함께 설명한다.

14. 기본 문법과 타입 시스템 — 해설

해설 1 — value type 복사.

출력

5
8

설명

- `int`는 **value type**이다. `int b = a;`는 `a`의 값을 `b`에 복사한다. 따라서 `b`는 `a`와 독립된 별개의 저장 공간을 갖는다.
- `b += 3;`은 `b`의 값만 8로 바꿀 뿐, `a`에는 영향을 주지 않는다.
- **value type**은 변수 자체가 데이터를 보관하며, 대입 시 비트 단위로 복제되어 독립적인 사본이 만들어진다. 함수에 전달할 때도 기본은 값 복사다.

해설 2 — implicit / explicit conversion.

원인: `double`은 정밀도/크기 모두 `int`보다 크고, 소수부가 손실될 수 있어 컴파일러가 암묵적 변환을 거부한다.

- **implicit conversion**: 손실이 없는 변환(예: `int` → `long`, `int` → `double`). 컴파일러가 자동으로 수행.
- **explicit conversion**: 손실 가능성이 있는 변환. 캐스트 연산자 (`T`)를 명시해야 한다.

수정 방법

```
int y = (int)x;           // (1) cast - 소수부는 절삭
int y = Convert.ToInt32(x); // (2) 반올림(banker's rounding) 적용
```

(추가) `(int)3.9`는 3, `Convert.ToInt32(3.5)`는 4(좌측 반올림: banker's rounding으로 정확히는 `3.5`→4, `2.5`→2)다.

해설 3 — boxing / unboxing.

출력

System.Int32
15

- **boxing 위치**: `object obj = 10;` — **value type**(`int`)이 **object**로 변환되며 **힙에 박스 객체**가 할당된다.
- **unboxing 위치**: `(int)obj` — 박스에서 값을 꺼낸다. 캐스트 타입이 박스 안의 실제 타입과 같아야 한다(다르면 `InvalidCastException`).

- **비용:** heap 할당 + GC 압력 + 캐시 미스. 루프 안의 boxing은 성능 문제의 흔한 원인이다. 제네릭 (`List<int>`)이 `ArrayList`보다 빠른 핵심 이유.

해설 4 — var vs dynamic.

항목	var	dynamic
타입 결정	컴파일 타임	런타임
정적 검사	가능	거의 불가
성능	일반 변수와 동일	DLR 오버헤드
안전성	컴파일 오류 발견	<code>RuntimeBinderException</code> 발생 가능

- `var x = 10;` 은 `int x = 10;` 과 동일하게 컴파일된다. 단순한 타입 추론 도우미.
- `dynamic`은 모든 멤버 접근을 런타임에 바인딩한다. COM/리플렉션/JSON 동적 객체 등 진짜 동적이 필요한 곳에만 쓰는 게 좋다.

해설 5 — 정수 overflow.

출력: -2147483648 (= `int.MinValue`)

- 기본적으로 산술 연산은 `unchecked` 컨텍스트에서 수행되며 **wrap-around**가 일어난다.
- `checked` 블록 안에서는 overflow 시 `OverflowException`이 발생한다.

```
checked
{
    int x = int.MaxValue + 1; // OverflowException
}
```

빌드 옵션(`/checked+`)으로 프로젝트 전체를 `checked`로 만들 수도 있다. 금융 계산 등 안전 중요한 코드는 `checked` 권장.

해설 6 — string immutability.

출력: Hello World

- `string`은 **불변(immutable)**이다. `s += " World"`는 기존 "Hello"를 수정하는 것이 아니라, 새 문자열 "Hello World"를 만들고 `s`가 그것을 가리키도록 바꾸는 것이다.
- "Hello" 객체는 누군가 참조하지 않으면 GC 대상이 된다.
- 반복적 결합(`for` 안에서 `s += ...`)은 매번 새 객체를 만들기 때문에 $O(n^2)$ 메모리 복사를 유발한다. → `StringBuilder` 사용.
- 불변성의 이점: 스레드 안전, 해시 캐싱(`string.GetHashCode`), 인터닝.

해설 7 — nullable value type.

출력

```
False
10
-1
```

- `int?`는 `Nullable<int>` 구조체의 별칭. `HasValue`와 `Value` 프로퍼티를 갖는다.
- `b ?? -1`: `b`가 `null`이 아니면 `b.Value`, `null`이면 `-1`. C#의 **null-coalescing operator**.
- `Nullable<T>`는 구조체이므로 박싱 규칙이 특수하다: `null`인 nullable을 박싱하면 `null` 참조가 된다 (빈 박스가 아님).

해설 8 — char vs string.

- char: **value type**, 2 바이트(UTF-16 code unit) 하나.
- string: **reference type**, 가변 길이 문자 시퀀스.

코드 결과: `c == s[0] → True`. `s[0]`은 char 타입이고 두 char를 비교한 것. 주의: surrogate pair(이모지 등)는 char 하나로 표현되지 않는다.

해설 9 — const vs readonly.

항목	const	readonly
값 결정 시점	컴파일 타임	런타임(생성자에서)
종류	컴파일 타임 상수만	임의의 표현식
인스턴스 다른 값	불가(암묵적 static)	가능
외부 어셈블리 변경 시	재컴파일 필요	자동 반영

위 코드

- (a) `const int A = DateTime.Now.Year;` — **불가능**. `DateTime.Now`는 런타임 값.
- (b) `readonly int B = DateTime.Now.Year;` — **가능**. 생성자(또는 필드 초기화 식)에서 한 번 설정.

라이브러리 공개 API에서 자주 변할 수 있는 상수는 `static readonly`가 더 안전하다(소비자 재컴파일 불필요).

해설 10 — nameof.

출력: field

- `nameof(field)`는 **컴파일 타임**에 식별자 이름을 문자열 리터럴 "field"로 치환한다.
- 단순 문자열 "field"보다 좋은 이유:
 - 리팩토링으로 변수명이 바뀌면 `nameof`는 자동으로 따라간다.
 - 컴파일러가 "그 이름이 실제로 존재하는지" 검사해 준다.
- 사용 예: `throw new ArgumentNullException(nameof(name));`, `INotifyPropertyChanged`의 프로퍼티명 전달.

15. 조건문과 반복문 — 해설

해설 11 — continue.

출력: 0 1 3 4

- `continue`: 현재 반복(iteration)만 건너뛰고 다음 반복으로 진행. 반복문 자체는 계속 실행.
- `break`: 반복문 전체를 즉시 종료하고 다음 문장으로 이동.
- `for`의 경우 `continue`는 증감식(`i++`)을 건너뛰지 않는다. 반면 `while`에서 `continue`를 쓰면서 증가 코드를 본문 끝에 두면 무한 루프에 빠질 수 있다.

해설 12 — 무한 루프.

- 문제: `n`을 갱신하는 코드가 없어 `n < 100`이 영원히 참 → 무한 출력.

- **원인:** 반복 종료 조건이 변수 변화에 의존하는데 그 변수를 갱신하지 않음.

수정

```
int n = 1;
while (n < 100)
{
    Console.WriteLine(n);
    n++; // 또는 n *= 2 등
}
```

또는 for로:

```
for (int n = 1; n < 100; n++) Console.WriteLine(n);
```

해설 13 — switch fall-through와 goto case.

출력

two

one

- 일반적으로 C#의 switch는 C와 달리 **fall-through**를 허용하지 않는다(case 끝에 break/return/throw/goto 필수).
- goto case 1;로 다른 case로 명시적 이동 가능.
- C에서는 case 2: 뒤 break가 없으면 자동으로 case 3:으로 떨어지는데, C#은 이를 컴파일 오류로 처리해 의도치 않은 fall-through 버그를 막는다.

해설 14 — switch expression.

- `_`는 **discard pattern** — “위의 어떤 패턴에도 매칭되지 않으면” 의미. default에 해당.
- switch expression은 **값을 반환**하며, **pattern matching**(범위, 타입, 분해 등)을 자연스럽게 표현한다.
- 전통적 switch는 문장이고 모든 case에 break 필요. switch expression은 식이고 콤마로 case를 구분, 더 짧고 함수형 스타일이다.

해설 15 — for / foreach / while.

- **for:** 반복 횟수가 정해져 있고 인덱스가 필요할 때.
- **foreach:** 컬렉션 순회. IEnumerable/IEnumerable<T>(혹은 GetEnumerator 패턴)를 구현한 것 이면 동작. 인덱스는 제공되지 않지만 의도가 명확하다.
- **while:** 종료 조건이 외부 상태에 의존하거나 반복 횟수가 미정일 때(파일 끝까지, 사용자 종료 입력까지 등).

```
while ((line = reader.ReadLine()) != null) { /* ... */ }
```

해설 16 — do-while.

- do-while은 본문을 **최소 한 번은 실행한 뒤** 조건을 검사. while은 검사 후 실행.
- 사용자 입력 검증처럼 “최소 한 번 시도하고 잘못되면 반복”하는 패턴에 적합.

```
int n;
do
{
    Console.Write("Enter positive: ");
} while (!int.TryParse(Console.ReadLine(), out n) || n <= 0);
```

컬렉션 순회는 foreach, 조건 미리 체크는 while이 더 적절.

해설 17 — 중첩 루프와 break.

출력

```
0,0
1,0
2,0
```

- break는 자신을 직접 감싸는 하나의 반복문만 종료한다. 위 코드에서 break는 안쪽 for만 빠져나오므로 바깥은 계속 진행.
- 바깥 루프까지 종료하려면:
 - 플래그 변수
 - 메서드로 추출 후 return
 - goto label; — 합법이지만 일반적으로 권장하지 않음.
- 함수형 스타일이라면 LINQ로 평탄화한 뒤 FirstOrDefault 등을 쓰는 방식도 가능.

해설 18 — type pattern + when.

출력: big int

- case int n when n > 10: — type pattern으로 x가 int임을 검사하면서 변수 n에 캐스트된 값을 바인딩, 추가로 when n > 10 조건이 참이어야 진입.
- when 절은 case 진입 가드다. 일반 if와 달리, 매칭 실패 시 다음 case로 자연스럽게 넘어간다.

16. 배열과 컬렉션 — 해설

해설 19 — 배열 vs List.

항목	T[] (배열)	List<T>
크기 변경	불가(재할당 필요)	가능(Add, Remove)
메모리	연속 블록, 고정 크기	내부 배열 + 길이; 가득 차면 두 배로 재할당
인덱스 접근	O(1)	O(1)
삽입/삭제(중간)	불편/비효율	O(n)
Add 평균 시간	—	분할 상황 O(1)
메모리 오버헤드	거의 없음	약간 더 큼

크기가 고정·성능 중요 → 배열. 동적 변화·편의성 → List<T>.

해설 20 — foreach 출력과 수정.

출력: 2 4 6

- foreach의 반복 변수는 읽기 전용이다. x = 0;은 컴파일 오류.
- 원소를 바꾸려면:

```
for (int i = 0; i < arr.Length; i++) arr[i] = arr[i] * 2;
```

또한 foreach 중에 컬렉션을 수정하면(Add/Remove) InvalidOperationException. 이는 enumerator의 version 검사 때문이다.

해설 21 — rectangular vs jagged.

	int[,], 다차원	int[][] 가변(jagged)
메모리	하나의 연속 블록	배열의 배열(행마다 별도 객체)
각 행 크기	동일해야 함	다르게 가능
인덱스 접근	a[i,j]	b[i][j]
성능	JIT가 다소 약함	일반적으로 더 빠름(독립 배열)

격자(체스판) → 다차원. 삼각형/희소 데이터 → jagged.

해설 22 — Dictionary.

출력

2

(예외: KeyNotFoundException)

- 같은 키에 인덱서로 값을 다시 넣으면 덮어쓴다(Add는 충돌 시 예외).
- 존재하지 않는 키 조회는 KeyNotFoundException 발생.
- 안전한 조회:

```
if (d.TryGetValue("B", out var v)) Console.WriteLine(v);
```

TryGetValue는 예외 비용 없이 분기 가능 — 핵심 경로에서 자주 쓰임.

해설 23 — HashSet.

출력: 3

- HashSet<T>은 중복 없는 컬렉션. 같은 원소를 두 번 넣어도 한 번만 저장됨.
- 내부적으로 해시 버킷 + 체이닝. Contains, Add, Remove 모두 평균 O(1).
- 사용자 정의 타입을 담으려면 Equals와 GetHashCode를 일관되게 오버라이드해야 한다.

해설 24 — Queue vs Stack.

출력: 1 3

- Queue<T>: FIFO (선입선출). Enqueue로 넣고 Dequeue로 가장 먼저 들어온 것 꺼냄.
- Stack<T>: LIFO (후입선출). Push로 넣고 Pop으로 가장 마지막에 들어온 것 꺼냄.
- BFS는 Queue, DFS는 Stack(또는 재귀).

해설 25 — Array.Sort vs List.Sort.

- 둘 다 내부적으로 introsort(quicksort + heapsort + insertion sort 혼합) 사용.
- 안정 정렬이 아니다. 동일 키 원소들의 상대 순서가 보존되지 않는다.
- 안정 정렬이 필요하면 LINQ OrderBy(merge sort 기반)를 사용.
- 사용자 정의 비교:

```
list.Sort((a, b) => a.Age.CompareTo(b.Age));
Array.Sort(arr, Comparer<int>.Create((a,b) => b.CompareTo(a))); // 내림차순
```

해설 26 — LinkedList vs List.

	List<T>	LinkedList<T>
구조	연속 배열	양방향 연결 노드
인덱스 접근	O(1)	O(n)
중간 삽입/삭제	O(n) (이동)	노드 위치를 알면 O(1)
메모리 오버헤드	적음	노드마다 prev/next 포인터
캐시 친화성	매우 좋음	나쁨

실무에선 List<T>가 거의 항상 더 빠르다(현대 CPU 캐시 효과 때문). LinkedList<T>는 매우 드물게 사용.

17. 메서드와 매개변수 — 해설

해설 27 — call by value.

출력: 5

- int는 value type. Change(a) 호출 시 a의 값이 매개변수 x로 복사된다.
- 메서드 안에서 x = 100;은 사본만 바꿀 뿐, 원본 a에 영향이 없다.
- C#에서 모든 기본 매개변수 전달은 값 복사다. reference type을 넘길 때는 참조가 복사되므로(참조 자체가 값) 객체의 멤버 변경은 보이지만 매개변수 재대입은 보이지 않는다.

해설 28 — ref / out / in.

	ref	out	in
호출 전 초기화	필요	불필요	필요
메서드 내 초기화	의무 아님	필수(모든 경로)	불가(읽기 전용)
의미	양방향 참조	출력 전용	입력 전용(복사 회피)

```
void Add(ref int x) { x += 10; }
void Init(out int x) { x = 42; }
double Length(in Vector v) => Math.Sqrt(v.X * v.X + v.Y * v.Y);
```

- out은 Tuple 등장 전 다중 반환의 대안. TryParse가 대표 예.
- in은 큰 struct(예: Matrix4x4)를 복사 없이 읽기 전용 참조로 넘길 때.

해설 29 — params.

- params는 가변 인자 배열. 호출 측이 임의 개수의 인자를 콤마로 나열할 수 있다.
- 컴파일러는 Sum(1,2,3)을 Sum(new int[] {1,2,3})으로 바꿔준다. 명시적 배열을 직접 넘기는 것도 가능.
- params는 메서드 시그니처의 **마지막** 매개변수에만 올 수 있다(이후 위치 인자를 어떻게 해석할지 모호해지므로).
- 성능에 민감한 경로에서 params 호출은 매번 배열 할당이 일어남에 유의(C# 12+의 params ReadOnlySpan<T>로 완화 가능).

해설 30 — optional / named arguments.**출력**

Hello, Alice!

Hello, Bob?

- default value 덕분에 호출 측이 매개변수 일부를 생략 가능.
- named argument는 순서를 무시하고 이름으로 인자를 지정 — 가독성 + 일부만 변경하기 편함.
- 오버로딩 대신 default 사용의 단점: default 값은 **호출 측 어셈블리에** 인라인된다(라이브러리 default 변경 시 재컴파일 필요).

해설 31 — 오버로딩 규칙.

- 시그니처는 이름 + 매개변수 타입/개수/수정자로 결정. 반환 타입은 포함되지 않는다.
- `void Foo(int x)` 두 개는 시그니처가 동일 → 충돌(컴파일 오류).
- `void Foo(int x, int y = 0)`은 `Foo(int)` 호출과 모호해질 수 있다 — 컴파일러는 덜 일반적인 매개변수가 적은 쪽을 우선 선택.
- 호출 시 일치성: exact > implicit conversion > params 확장 순으로 우선.

해설 32 — extension method.

- 정의 규칙: **static** 클래스 내부의 **static** 메서드, 첫 매개변수 앞에 **this**.
- 컴파일 후에는 일반 static 메서드 호출과 동일(`StringExt.IsEmpty(s)`).
- 인스턴스 메서드처럼 보이는 문법 설탕(*syntactic sugar*)이지만, 동적 디스패치가 아니며 **컴파일 타입에 결정된다**.
- 자주 쓰는 곳: LINQ(`Where`, `Select` 등은 모두 확장 메서드), 도메인 모델에 메서드 추가하기 곤란할 때.

해설 33 — local function.

- 메서드 내부에 정의된 메서드. 외부 스코프 변수를 캡처할 수 있다.
- 람다와 비교:
 - **할당 없음**: 람다는 delegate 객체로 변환되며 캡처 변수는 클로저 클래스 인스턴스를 만들. local function은 일반 메서드처럼 호출되고 캡처도 더 효율적(struct 클로저).
 - **재귀 가능**: 람다는 자기 자신을 깔끔히 참조 못 함(forward declaration 필요).
 - 이름 기반 디버깅 용이.
- helper 한 번만 쓰는데 사용 범위가 좁을 때, 또는 iterator/async의 인자 검증을 분리할 때 유용.

해설 34 — expression-bodied member.

- `=>` 뒤에 단일 식을 두는 간결한 구문. 메서드, 프로퍼티, 생성자, 인덱서, finalizer 등에 모두 적용 가능.
- 컴파일 결과는 전통적인 본문 `{ return ...; }` 과 동일.
- 짧은 read-only 프로퍼티/계산 메서드의 가독성을 크게 향상시킨다.

18. 객체지향 프로그래밍 — 해설

해설 35 — class vs struct.

	class	struct
분류	reference type	value type
메모리	주로 heap	주로 stack 또는 포함 객체 안
대입/전달	참조 복사	값 전체 복사
기본값	null	모든 필드 0/default
상속	가능	다른 struct 상속 불가(인터페이스만)
사용 시점	동일성/식별성 있는 객체	작고 불변에 가까운 값 묶음

일반론: 16바이트 이하 + 불변 + 자주 단명이라면 struct, 그 외에는 class. 큰 struct 복사는 비용이 크고, mutable struct는 미묘한 버그의 원인이 되므로 피하라.

해설 36 — reference type 공유.

출력: Bob

- Person은 class(reference type). `p2 = p1;`은 같은 객체에 대한 또 다른 참조를 만든다. `p2.Name = "Bob"`은 `p1`이 가리키는 동일 객체의 필드를 바꾼 것.
- shallow copy: 객체 자체는 복사되지만 그 객체가 가진 참조 멤버는 같은 대상을 가리킨다.
- deep copy: 참조 멤버까지 재귀적으로 복제. 직접 구현하거나 직렬화/record with/clone 패턴 등을 사용.

해설 37 — 상속.

장점: 코드 재사용, “is-a” 모델링, 다형성 지원. 단점: 부모-자식 강결합, 유연성 저하, 깊은 계층은 이해 어려움 → “상속보다 합성” 격언.

- base class: 공통 멤버를 제공하는 부모 클래스.
- derived class: base를 확장/특화하는 자식 클래스.
- virtual: 자식에서 재정의 허용을 base에서 선언.
- override: 자식에서 부모의 virtual 메서드를 새로 정의.
- polymorphism: 같은 참조 타입의 메서드 호출이 실제 인스턴스 타입에 따라 다르게 동작.

해설 38 — 접근 제한자.

접근자	같은 클래스	같은 어셈블리	자식(다른 어셈블리)	다른 어셈블리
public	O	O	O	O
private	O	X	X	X
protected	O	(자식 통해)	O	X
internal	O	O	X	X
protected internal	O	O	O	X
private protected	O	(자식 통해)	X	X
file (C# 11+)	동일 파일 내만	—	—	—

가능한 가장 좁은 가시성으로 시작하는 게 좋다.

해설 39 — 생성자 체이닝.

- `this(...)`: 같은 클래스의 다른 생성자를 먼저 호출.
- `base(...)`: 부모 클래스 생성자를 호출.
- **호출 순서**: 부모 생성자가 항상 먼저 실행 → 그 다음 자식 생성자.
- 명시하지 않으면 `base()`(매개변수 없는 부모 생성자)가 자동 호출. 부모에 매개변수 없는 생성자가 없으면 명시적으로 `base(...)` 필요.

해설 40 — `static class / member`.

- `static` 클래스: 인스턴스 생성 불가. 모든 멤버가 `static`이어야 함. 상속 불가, 봉인된 `abstract`로 컴파일됨.
- `static` 멤버: 인스턴스 없이 클래스 단위로 존재.
- `static` 생성자: 해당 타입이 처음 사용되기 직전에 단 한 번 실행. 매개변수 불가, 접근 제한자 불가.
- 유틸리티(예: `Math`)나 확장 메서드 컨테이너에 사용.

해설 41 — `partial class`.

- 같은 클래스 정의를 여러 파일로 분할. 컴파일러가 합쳐서 하나로 처리.
- 활용: WinForms/WPF의 디자이너 코드, Roslyn source generator, Entity Framework reverse engineering 등 **자동 생성 코드와 사람이 쓰는 코드의 분리**.
- `partial` 메서드: 선언만 두고 구현은 다른 `partial` 파일에 둘 수 있다(없으면 호출이 제거됨).

해설 42 — `sealed`.

- `sealed class`: 더 이상 상속 불가.
- `sealed override`: 자식에서 부모의 `virtual`을 `override`하면서 손자가 다시 `override`하는 것을 막음.
- 효과:
 - 의도 표현(이 클래스는 확장점이 아니다).
 - **JIT 최적화**: `virtual` 호출을 정적 디스패치로 바꿀 수 있어 더 빠를 수 있다.
 - 보안/계약 관점에서 일관성 확보.

해설 43 — `auto-property`.

- `public string Name { get; set; }` — 컴파일러가 백킹 필드와 `get/set`을 자동 생성.
- `public int Age { get; private set; }` — 외부에선 읽기 전용, 클래스 내부에서만 변경 가능.
- `public string FullName => $"{Name} ({Age})";` — 읽기 전용 계산 프로퍼티. 매번 평가됨.
- `init` 접근자(C# 9+): 생성 직후의 초기화에서만 `set` 허용 → 사실상 불변 객체.

해설 44 — `nested class`.

- 외부 클래스의 `private` 멤버에도 접근 가능(C# `nested class`는 자바와 달리 자동으로 `outer` 인스턴스 참조를 갖지 않는다; `outer` 인스턴스가 필요하면 명시적으로 전달).

- 사용 시점: outer 클래스와 강하게 결합되어 외부에는 노출할 필요 없는 helper(예: enumerator 구현).

해설 45 — 객체 초기화자.

- 동작 순서: 생성자 호출 → 반환된 인스턴스에 대해 순서대로 프로퍼티/필드 setter 호출.
- 초기화 가능 대상: 접근 가능한 **public/internal setter**를 가진 프로퍼티나 필드. **init** 접근자만 있는 프로퍼티도 가능.
- 가독성: 매개변수 많은 생성자 대신 의도가 분명한 멤버 단위 초기화.
- 주의: 초기화자가 던진 예외가 발생해도 생성자는 이미 완료된 상태이므로 partial 초기화된 객체가 GC 대상이 된다.

해설 46 — record.

- **value-based equality**: 같은 타입 + 같은 멤버 값이면 ==가 true. 일반 class는 참조 동일성.
- `record Point(int X, int Y);` — primary constructor + auto property + Equals/GetHashCode/ToString 을 컴파일러가 생성.
- with 식: 일부 필드만 바꾼 **새 인스턴스**를 생성(비파괴 변경).

```
| var p3 = p1 with { X = 99 };
```

record struct: value type 버전. 작고 불변인 값을 표현. 기본 비교 동작은 동일하지만 stack 할당.

19. 인터페이스와 추상화 — 해설

해설 47 — abstract class vs interface.

	abstract class	interface
다중 상속	단일	다중 가능
구현	일부/전체 가능	C# 8.0+ default 구현 가능, 기본은 선언만
필드	가질 수 있음	없음(상수 정도)
생성자	있음	없음
사용 의미	“is-a” + 공유 구현	“can do” 능력 제약

기본은 interface로 설계 → 공통 구현이 많아지면 abstract class로 보강.

해설 48 — 다형성.

출력: Woof

- 변수 a의 컴파일 타입은 Animal, 런타임 타입은 Dog.
- virtual 메서드 호출은 vtable(가상 함수 테이블)을 통해 런타임 타입에 따라 정확한 구현으로 분기 → 동적 디스패치.
- new 키워드로 부모 메서드를 “숨기기”만 한 경우는 정적 디스패치 → 변수의 컴파일 타입에 따라 다르게 동작(주의).

해설 49 — default interface method.

- C# 8.0부터 인터페이스가 기본 구현을 가질 수 있다.

- 의의: 기존 인터페이스에 메서드를 추가해도 **모든 구현체를 깨 필요 없다**(API 진화).
- 다이아몬드 문제: 두 인터페이스에서 같은 시그니처의 default 구현이 충돌하면, 구현 클래스에서 명시적으로 재정의(또는 명시적 구현)해야 함.
- 한계: 필드 없음, 인스턴스 상태 없음, “구현 상속”의 대체는 아님.

해설 50 — 명시적 인터페이스 구현.

```
IA a = new Foo(); a.Show(); // "IA"
IB b = new Foo(); b.Show(); // "IB"
((Foo)new Foo()).Show();    // 컴파일 오류 - public 멤버 없음
```

- 명시적 구현 메서드는 인터페이스 타입을 통해서만 호출 가능. 클래스 인스턴스 인터페이스에선 보이지 않음.
- 같은 시그니처의 메서드가 두 인터페이스에서 충돌할 때 분리 가능.
- 또는 “공개 표면적은 줄이고 인터페이스 사용자에게만 노출”하고 싶을 때 유용.
- 접근 제한자를 붙일 수 없다(default: 인터페이스에 한해 접근 가능).

해설 51 — IEnumerable / IEnumerator.

- foreach (var x in xs) 가 동작하려면 xs가 GetEnumerator() 메서드를 갖고, 반환된 enumerator에 MoveNext()와 Current가 있으면 된다(엄밀히는 인터페이스 구현 불필수, 패턴 기반).
- IEnumerable<T>: “열거 가능한 것”. IEnumerator<T>: “지금 어디까지 봤는지를 기억하는 커서”.
- yield return은 컴파일러가 자동으로 enumerator 상태 머신을 생성해 준다. 호출 시 코드가 한 번에 실행되는 것이 아니라, MoveNext 호출마다 다음 yield까지 진행한다.

```
IEnumerable<int> Numbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

해설 52 — IDisposable / using.

using (var x = new T()) { ... } →

```
var x = new T();
try { /* ... */ }
finally { if (x != null) ((IDisposable)x).Dispose(); }
```

- Dispose는 **unmanaged resource**(파일 핸들, 소켓, DB 연결 등)나 결정적 해제가 필요한 자원을 즉시 정리.
- C# 8.0의 using 선언:

```
using var x = new FileStream(/* ... */);
// 스코프 끝에서 자동 Dispose
```

비동기 자원은 IAsyncDisposable + await using.

해설 53 — IComparable vs IComparer.

- IComparable<T> (int CompareTo(T other)): 자기 자신을 다른 인스턴스와 비교 — 타입 자체의 자연 순서.

- `IComparer<T> (int Compare(T x, T y))`: **외부 비교자**. 같은 타입에 대해 여러 정렬 기준을 만들고 싶을 때.
- `Array.Sort/List.Sort/OrderBy`는 둘 다 받을 수 있는 오버로드를 가짐.
- `IComparable`은 안정성과 무관 — 정렬 알고리즘 자체의 성질이다.

해설 54 — ISP.

`IWorker`의 문제: 로봇 작업자는 `Eat/Sleep`이 의미 없는데도 구현해야 한다 → `NotImplementedException` 던지기? 결국 LSP도 깨짐. **분리 예**

```
interface IWorkable { void Work(); }
interface IFeedable { void Eat(); }
interface ISleepable { void Sleep(); }

class Human : IWorkable, IFeedable, ISleepable { /* ... */ }
class Robot : IWorkable { /* ... */ }
```

“클라이언트는 자신이 사용하지 않는 메서드에 의존해서는 안 된다.”

20. 예외 처리 — 해설

해설 55 — `DivideByZeroException`.

`10 / 0` (정수 나눗셈)에서 `System.DivideByZeroException`. 참고: `double / 0.0`은 예외가 아니라 `Infinity` 또는 `NaN`(IEEE-754 규칙).

```
try
{
    int y = 10 / x;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"오류: {ex.Message}");
}
finally
{
    Console.WriteLine("정리 작업"); // 정상/예외 어느 경우에도 실행
}
```

`finally`는 자원 해제, 잠금 해제, 로그 출력 같은 **무조건 실행**되어야 하는 후처리에 적합.

해설 56 — 예외 남용 금지.

- 성능: 예외 발생/캐치는 비싸다(스택 워크, 메타데이터). 정상 흐름의 분기에 쓰면 안 됨.
- 가독성: `try-catch`가 정상 로직처럼 자리잡으면 흐름을 따라가기 어렵다.
- 논리 오류 은폐: `catch (Exception)`으로 모든 예외를 삼키면 진짜 버그가 숨겨진다(“swallowed exception”).
- 원칙: 예측 가능한 분기는 `if`로, 예외적이고 회복 어려운 상황만 예외로. `TryParse`처럼 예외 없는 시도 `API`를 우선.

해설 57 — `throw` vs `throw ex`.

- `throw;` — 잡은 예외를 그대로 재던짐. 원래 `stack trace` 유지.
- `throw ex;` — `ex`를 새로 던지는 셈이라 `stack trace`가 현재 `catch` 위치부터 재설정. 디버깅에 매우 불리.

- 바람직: `throw`; 또는 `throw new MyException("context", ex);` (`InnerException`으로 원본 보존).

해설 58 — 커스텀 예외.

- 상속: 일반적으로 `Exception` 또는 그 하위(예: `ApplicationException` — 지금은 권장하지 않음). 도메인 오류면 `Exception` 직접 상속이 무난.
- 권장 생성자 4개:

```
public InvalidScoreException() { }
public InvalidScoreException(string msg) : base(msg) { }
public InvalidScoreException(string msg, Exception inner) : base(msg, inner) { }
// (legacy) protected InvalidScoreException(SerializationInfo, StreamingContext) { }
```

추가 데이터(Score 등)를 프로퍼티로 노출하여 호출자가 처리 시 정확한 정보 접근 가능.

해설 59 — exception filter (when).

- `catch (T ex) when (cond):` `cond`가 참일 때만 `catch` 진입. 거짓이면 다른 `catch` / 상위로 전파 — 그리고 `stack`을 풀지 않는다.
- 일반 `if`로 `catch` 안에서 검사 후 `throw`; 한 것과의 차이: `filter`는 `stack walk`이 일어나기 전에 평가되므로, 디버거가 던진 지점의 스택을 그대로 본다.
- 사용: 특정 HTTP 상태 코드, 특정 예러 코드, 특정 예외 메시지 패턴 등으로 분기.

해설 60 — try-finally / using.

- `try-finally`에서 `try` 본문 도중 예외가 나도 `finally`는 실행 → `conn.Close()` 보장.
- `IDisposable`을 구현한 타입이라면 `using`이 더 간결.

```
using var conn = OpenConnection();
DoWork(conn);
```

자원 누수: 파일 핸들, 소켓, DB 연결 등 운영체제 핸들이 회수되지 않아 누적되면 시스템 자원 고갈로 이어진다. GC는 `finalizer`가 있어도 언제 실행될지 보장하지 않는다.

21. 제네릭 — 해설

해설 61 — 제네릭 장점.

`List<int>` vs `ArrayList`:

- 타입 안전성:** `List<int>`에는 `int`만 들어간다. `ArrayList`는 `object`이므로 컴파일러가 잘못된 타입 삽입을 막지 못함.
- boxing 회피:** `ArrayList` 안의 `int`는 모두 박싱된 `object`. `List<int>`는 내부 배열이 `int[]` → heap 할당 0개.
- 재사용성:** 같은 `List<T>` 정의를 `int`, `string`, 사용자 타입 등에 그대로 사용.
- API 명확성:** `List<int>`에서 꺼낸 값은 `int` — 캐스트 불필요.

해설 62 — 제네릭 제약.

- `class` — `T`는 참조 타입(또는 nullable reference).

- `struct` — T는 (non-nullable) 값 타입.
- `new()` — 매개변수 없는 `public` 생성자가 있어야 함(`new T()` 가능).
- `interface` 이름 — 해당 인터페이스 구현.
- `base class` 이름 — 해당 클래스 또는 그 하위.
- `notnull` — null 불가 타입.
- `unmanaged` — 참조를 포함하지 않는 값 타입(`blittable`). `Span/interop`에서 사용.

여러 제약은 콤마로 나열, `class/struct`가 먼저, `new()`는 마지막.

해설 63 — 공변 / 반공변.

- `IEnumerable<out T>` — T를 출력으로만 쓰므로, `string`이 `object`인 곳에 자연스럽게 대입 가능.
- `Action<in T>` — T를 입력으로만 쓰므로, `Action<object>`를 `Action<string>`에 대입 가능(`object` 처리기는 `string`도 처리 가능).
- `List<T>`는 `in/out` 모두에 T가 등장(`Add`, `IndexOf`) → 불변.
- 배열의 공변: `string[] → object[]`은 합법이지만 런타임에 `ArrayTypeMismatchException`이 날 수 있어 위험. C# 초기 자바 호환을 위해 도입된 역사적 결정으로 평가됨.

해설 64 — 제네릭 메서드 vs 클래스.

- 제네릭 메서드: 호출 인자로부터 타입을 추론 가능(`First(new[] {1,2,3})`).
- 제네릭 클래스: 인스턴스화 시 타입을 명시(`new Box<int>()`).
- 클래스 전체가 T에 의존하지 않고 한 메서드만 T가 필요하다면 메서드만 제네릭으로 선언하는 게 좋다 (클라이언트가 클래스 매개변수를 지정할 필요가 없어진다).

해설 65 — 컬렉션 초기화자.

- 동작:
 - 기본 생성자(또는 지정 생성자) 호출
 - 각 원소에 대해 `Add(...)` 호출
- 따라서 `IEnumerable` 구현 + 적절한 `Add` 메서드만 있으면 사용자 정의 컬렉션에도 적용 가능.

```
class MyMap : IEnumerable
{
    public void Add(string key, int value) { /* ... */ }
    public IEnumerator GetEnumerator() => { /* ... */; }
}

var m = new MyMap { {"A", 1}, {"B", 2} };
```

Dictionary 초기화 문법 두 가지:

```
new Dictionary<string,int> { {"A", 1} }; // Add("A", 1)
new Dictionary<string,int> { ["A"] = 1 }; // indexer
```

전자는 중복 키에서 `ArgumentException`, 후자는 덮어쓴다.

해설 66 — 동시성 컬렉션.

- 일반 Dictionary<K,V>를 멀티스레드에서 동시에 수정/조회하면 내부 상태가 깨져 **무한 루프**, **null 참조**, **데이터 손상** 등이 발생할 수 있다.
- ConcurrentDictionary: 세분화된 잠금(혹은 lock-free)으로 Add/TryUpdate/AddOrUpdate 등 원자적 연산 제공.
- ConcurrentQueue/Stack: lock-free 큐/스택.
- BlockingCollection<T>: 생산자-소비자 패턴에 적합. 버퍼가 비어 있으면 소비자 블록, 가득 차면 생산자 블록.
- 단순 lock보다: 잠금 범위가 짧아 처리량 ↑, 데드락 위험 ↓. 단, 연산 단위의 원자성만 보장 — 연속된 두 호출의 합성은 여전히 호출자가 보장해야 함.

22. LINQ — 해설

해설 67 — Where/Select.

- Where(predicate): 조건을 만족하는 원소만 통과시키는 필터.
- Select(projector): 원소를 변환(map). 결과 타입이 달라질 수 있다.
- **deferred execution**: Where/Select 호출 시점에는 아무 연산도 일어나지 않는다. 반복(foreach, ToList 등)될 때 비로소 원소가 흐른다. 그래서 큰 시퀀스에 여러 LINQ 연산자를 체이닝해도 중간 컬렉션이 생기지 않음.

해설 68 — LINQ 장단점.

장점

- 의도가 코드에 자연스럽게 드러남(필터/투영/그룹/정렬).
- 함수형 스타일로 부작용 적음.
- 같은 문법으로 메모리/DB/XML 등 다양한 데이터 소스 다룸.

단점

- 무심코 ToList 여러 번 호출 시 메모리/시간 낭비.
- deferred execution을 모르면 “같은 query를 두 번 돌리면 또 평가됨” 같은 함정.
- 디버깅 시 중간값 확인이 까다로움(브레이크포인트 잡기 어려움 — .ToList()로 끊거나 LINQPad/디버거 visualizer).
- 비싼 람다나 N+1 쿼리(IQueryable) 같은 함정.

해설 69 — deferred vs immediate.

- q = numbers.Where(...) 시점엔 실행되지 않음. 컬렉션이 질의 그 자체.
- 그 사이 numbers.Add(100)이 일어나면 foreach 시 100도 포함될 수 있다(원본을 평가하기 때문).
- ToList()/ToArray()/Count()/First() 등은 **즉시 실행**해 스냅샷을 만든다.
- 부작용 위험: 원본 변경 + deferred 평가의 조합이 예측 불가능한 결과를 만들 수 있음 → 종종 query 직후 .ToList()로 고정.

해설 70 — GroupBy / Aggregate.

- `GroupBy(keySelector) → IEnumerable<IGrouping<TKey, TElement>>`. 각 `IGrouping`은 Key와 그 그룹의 원소들을 갖는 시퀀스.

```
foreach (var g in byLength)
{
    Console.WriteLine($"{g.Key}: {string.Join(", ", g)}");
}
```

- `Aggregate(seed, (acc, x) => ...)`: 누적자에 원소를 차례로 적용해 단일 값으로 fold.
- `Sum/Min/Max/Count/Average` 등은 `Aggregate`의 특화 케이스.

해설 71 — Join.

- SQL INNER JOIN과 의미 동일: 두 시퀀스를 키 동등 조건으로 매칭.
- `inner join` = 매칭되는 짝만. `left join`은 LINQ에선 `GroupJoin + SelectMany + DefaultIfEmpty`로 표현.

```
var leftJoin =
    from c in customers
    join o in orders on c.Id equals o.CustomerId into oc
    from o in oc.DefaultIfEmpty()
    select new { c.Name, Amount = o?.Amount ?? 0 };
```

`GroupJoin`은 외부 원소마다 내부 원소들의 시퀀스를 한 묶음으로 가져오는 연산.

해설 72 — ToList / ToArray / ToDictionary / ToHashSet.

- 모두 즉시 실행: 시퀀스를 끝까지 순회해 새 자료구조에 담는다.
- 메모리: 원본 + 사본 두 벌이 잠시 공존.
- 사용 주의:
 - 여러 번 순회할 거면 `ToList` 등으로 한 번만 평가하자.
 - 한 번만 순회할 거면 굳이 `materialize`하지 말자.
 - 무한 시퀀스(`Generate`, `Enumerable.Range(0, int.MaxValue)`)에 호출 금지.

해설 73 — method vs query syntax.

- 둘은 컴파일 결과가 동일한 메서드 호출 체인. `query syntax`는 일부 패턴(`Where`, `Select`, `Join`, `GroupBy`, `Let`, `OrderBy`)에 한해 컴파일러가 `method` 호출로 변환.
- 표현력: `SkipWhile`, `Zip`, `Aggregate`, `Distinct` 등은 `query syntax`에 없다 → `method syntax` 필요.
- 실무에서는 `method syntax`가 더 일반적. 복잡한 `join/group by/let`이 많은 코드는 `query syntax`가 가독성에 유리할 수 있다.

해설 74 — IEnumerable vs IQueryable.

- `IEnumerable<T>`: in-memory iteration. 람다는 컴파일된 delegate.
- `IQueryable<T>`: 람다가 `expression tree`로 캡처되고, provider(예: EF Core)가 이를 분석해 원격(SQL 등)으로 번역.
- 위험한 함정: `IQueryable`을 `IEnumerable`로 받는 순간(예: 메서드 매개변수 타입) 이후의 `Where/Select`는 클라이언트 측 평가가 된다 → 전체 데이터 가져오기.
- EF Core에서 가능한 한 `IQueryable`로 끝까지 가져가고 마지막에 `ToList` 같은 `materialize`.

23. 비동기 프로그래밍 — 해설

해설 75 — `async / await`.

- 목적: I/O 대기 시간 동안 스레드를 점유하지 않으면서 코드를 동기 스타일로 작성.
- `thread`와의 관계: `async`는 새 스레드를 만들지 않는다. `await` 지점에서 메서드가 일시 중지되고, 그 콜백이 (보통) 동일/별도 스레드에서 재개된다.
- UI freeze 해결: 동기 I/O 호출은 UI 스레드를 막아 화면이 멈춘다. `await`를 쓰면 UI 스레드는 즉시 메시지 루프로 돌아가고, 결과가 준비되면 자동으로 UI 스레드로 복귀해 이어 실행.

해설 76 — `async` 실행 흐름.

- `Test()` 호출 → “A” 출력 → `Task.Delay(1000)`이 시작되고, 여기서 `Test()`는 호출자에게 `Task`를 반환한다(즉시 반환). 호출자는 그 사이 다른 일 가능.
- 1초 뒤 타이머 콜백이 큐에 들어가면 적절한 스레드(`SynchronizationContext`가 있으면 그 컨텍스트의 스레드, 없으면 `thread pool`)에서 “B” 출력.
- 컴파일러는 `async` 메서드를 `state machine`(`MoveNext` 패턴)으로 변환. 각 `await`는 상태 분기점이 된다.

해설 77 — `Task vs Thread`.

- `Thread`: 운영체제 스레드 직접 추상화. 비용 큼, lifetime 직접 관리.
- `Task`: 작업(*work*) 추상화. 결과/진행/완료를 표현. 보통 `thread pool` 위에서 실행.
- `async` 메서드는 기본적으로 새 스레드를 만들지 않는다. 단순히 `await` 지점마다 콜백을 등록할 뿐. `Task.Run(...)`처럼 명시적 위임만이 `thread pool`로 보낸다.

해설 78 — `WhenAll / WhenAny`.

- `WhenAll`: 모든 `Task`가 완료될 때까지 대기. 결과는 배열(또는 `void`). 여러 작업을 동시에 시작해 총 대기 시간을 단축.
- `WhenAny`: 가장 먼저 완료된 `Task` 하나를 돌려줌. 타임아웃, 경합 처리 등에 사용.
- 예외 처리: `WhenAll`에서 여러 `Task`가 실패하면 첫 예외만 `await`로 다시 던져지지만 `Task.Exception`에는 `AggregateException`으로 모두 보존.

```
var completed = await Task.WhenAny(work, Task.Delay(timeout));
if (completed != work) throw new TimeoutException();
```

해설 79 — `ConfigureAwait(false)`.

- 기본 `await`는 원래의 `SynchronizationContext/TaskScheduler`로 복귀한다 → UI 코드에서 UI 스레드로 자동 복귀.
- `ConfigureAwait(false)`는 “복귀할 필요 없음”이라고 지시 → 콜백이 `thread pool` 스레드에서 그대로 실행. 비용 감소 + UI 컨텍스트 경합 데드락 회피.
- 라이브러리 코드: UI 컨텍스트를 알 필요가 없으므로 항상 `ConfigureAwait(false)` 권장.
- 앱 코드(UI/뷰모델): UI 객체에 접근하려면 컨텍스트 복귀가 필요하므로 기본값 유지.

- ASP.NET Core는 SynchronizationContext가 없어 큰 차이는 없지만 라이브러리 호환 차원에서 여전히 권장.

해설 80 — `async void`.

- 위험:
 - 호출자가 Task를 받지 못하므로 완료/예외를 기다릴 수 없다.
 - 예외가 SynchronizationContext로 던져져 보통 프로세스 종료를 유발.
 - 단위 테스트도 비동기 완료를 보장하기 어려움.
- 허용되는 경우: 이벤트 핸들러(예: `button.Click += async (s,e) => ...`) — 시그니처가 void 여야 하기 때문.

해설 81 — `CancellationToken`.

- C#의 취소는 협력적(cooperative)이다. 토큰을 받은 코드가 주기적으로 `token.IsCancellationRequested`를 확인하거나 `token.ThrowIfCancellationRequested()`를 호출해 중단해야 한다.
- `Task.Delay(ms, token)`처럼 토큰을 인자로 받는 비동기 메서드는 취소 시 즉시 `OperationCanceledException`(는 `TaskCanceledException`)으로 종료.
- `CancellationTokenSource`로 토큰을 만들고 `Cancel()`로 신호. 타임아웃 결합(`CancellationTokenSource.CreateLinkedTokenSource`)

해설 82 — `SynchronizationContext`.

- “이 콜백을 어디서 실행할까”를 결정하는 추상화.
- WinForms/WPF: UI 스레드 메시지 루프에 콜백을 보낸다 → `await` 후 UI 스레드 복귀.
- ASP.NET Core: 컨텍스트가 없음(`SynchronizationContext.Current == null`) → `ConfigureAwait(false)` 여부의 의미가 약함, deadlock 위험도 작음.
- WinForms 코드를 .NET 6 콘솔/ASP.NET Core로 이식할 때, 컨텍스트 의존 가정이 깨지는 버그가 흔히 나온다.

24. 메모리와 런타임 — 해설

해설 83 — GC.

- CLR은 managed heap에서 객체를 할당하고, 더 이상 참조되지 않는 객체를 자동 회수.
- 동작 단계: (1) **mark**: 루트(스택, 정적, 핸들)에서 시작해 도달 가능한 객체에 표시. (2) **sweep/compact**: 표시되지 않은 객체 회수, 살아남은 객체 압축.
- **stop-the-world**: 일부 단계에서 모든 managed 스레드를 잠시 멈춤. 최신 .NET의 *concurrent/background GC*는 일부 단계를 백그라운드로 돌려 정지 시간을 최소화.
- 개발자 효과: 메모리 해제 코드 불필요, 그러나 (a) 결정적 해제는 `IDisposable/using`, (b) 큰 객체나 LOH는 별도 고려.

해설 84 — `stack vs heap`.

	stack	heap
저장 대상	메서드 지역(값 타입) + 매개변수 + 호출 정보	객체(class 인스턴스, 박싱된 값 등)
할당/해제	매우 빠름(포인터 증감)	더 비쌈(자유 영역 관리)
lifetime	메서드 종료 시 자동 회수	GC가 회수
크기	보통 1~4MB, 작음	매우 큼(가상 메모리 한도)
동시성	스레드별 분리	공유(따라서 GC 동기화 필요)

“struct는 항상 stack”은 단순화된 표현이다. struct가 class 필드로 들어가면 그 객체의 일부로 *heap*에 자리잡는다. 박싱된 struct도 heap.

해설 85 — 세대별 GC.

- 가설: “대부분의 객체는 짧게 살고(weak generational hypothesis), 살아남은 객체는 오래 산다.”
- **Gen 0**: 새로 할당된 객체. 크기 작고 자주 수집(빠름).
- **Gen 1**: Gen 0에서 살아남은 객체. 중간 영역.
- **Gen 2**: 장수 객체. 비싸지만 자주 수집할 필요 없음.
- Gen 0 수집은 살아 있는 객체가 적어 매우 빠르며, 대부분의 회수가 여기서 일어난다.

해설 86 — IDisposable vs finalizer.

	Dispose	finalizer (~Foo())
호출 시점	결정적(개발자가 호출/using)	비결정적(GC가 적당히)
사용 의도	일반적인 자원 해제	잊혔을 때 마지막 보험
비용	낮음	finalize queue 경유, Gen 승격 비용 큼

Disposable Pattern (unmanaged resource 포함 시):

```
class Resource : IDisposable
{
    private IntPtr _handle;
    private bool _disposed;

    ~Resource() => Dispose(false);

    public void Dispose() { Dispose(true); GC.SuppressFinalize(this); }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;
        if (disposing) { /* managed dispose */ }
        // free unmanaged: _handle, ...
        _disposed = true;
    }
}
```

순수 managed 자원만 다룬다면 finalizer는 만들지 말 것(비용만 추가).

해설 87 — LOH.

- 약 85,000 바이트 이상의 객체는 LOH에 할당.
- 차이:
 - 일반 heap은 압축(compaction)되지만, LOH는 기본적으로 압축하지 않음(.NET 4.5.1+에서 명시적 요청 시만).

- **단편화** 위험: 큰 배열을 자주 만들고 버리면 LOH에 구멍이 늘어 OOM 가능.
- 권장: 큰 버퍼는 풀링(ArrayPool<T>), 큰 배열은 재사용.
- 큰 객체는 Gen 2 비용에 가까운 수집 비용을 가진다.

해설 88 — Span / Memory.

- Span<T>: 연속 메모리(배열, stackalloc, 비관리 메모리)에 대한 **slice** 참조. zero-copy.
- ref struct이므로 **heap에 둘 수 없다**(필드, 박싱, async 메서드 안에서 직접 사용 불가).
- Memory<T>: span의 자매. heap에 둘 수 있고, async 메서드에서도 사용 가능. 필요할 때 memory.Span으로 변환.
- 의의: 파서/네트워크 코드 등에서 **불필요한 할당과 복사 없이** 부분 시퀀스를 다룰 수 있어 성능이 극적으로 개선.

```
ReadOnlySpan<char> view = source.AsSpan().Slice(start, len);
```

25. 고급 / 함수형 / 위임 — 해설

해설 89 — null 참조.

- list.Count 호출에서 list가 null → **NullReferenceException**.
- C# 8.0의 **nullable reference type**: List<int>?로 명시. 컴파일러가 null 가능성을 추적해 경고.
- 안전 코드:

```
Console.WriteLine(list?.Count ?? 0);
if (list != null) Console.WriteLine(list.Count);
```

가능한 한 컬렉션은 빈 컬렉션으로 초기화(new List<int>()), null을 흘리지 말 것.

해설 90 — delegate.

- **타입 안전한 함수 포인터**. 메서드 시그니처를 타입으로 표현하고, 인스턴스화하면 그 시그니처를 만족하는 메서드를 담을 수 있는 객체.
- 다중 호출(+=) 가능 — invocation list 보존. 호출 시 모두 차례로 실행.
- **event**: delegate에 캡슐화를 더한 것. 외부에서 +=/-=만 가능.
- **람다**는 delegate(또는 expression tree)로 변환되는 **익명 메서드**. 캡처가 있으면 컴파일러가 closure 클래스 생성.

해설 91 — Func / 람다.

- Func<TArg, TResult>: BCL 제공 제네릭 delegate. 인자 1개, 반환 1개.
- x => x * x: int x를 받아 x*x 반환하는 람다 → Func<int, int>에 부합.
- delegate 타입의 변수에 람다를 할당하면 컴파일러가 **익명 메서드**를 만들어 delegate로 래핑.
- Action<T>(반환 없음), Predicate<T>(bool 반환)도 흔히 쓰임.

해설 92 — closure.

3 3 3

- 람다는 변수 자체(스토키지)를 캡처한다. 기존(C# <5)의 for 변수 *i*는 루프 전체에서 **하나**의 변수 → 모든 람다가 동일한 슬롯을 가리킨다 → 출력 시점에 *i* == 3.
- C# 5부터 **foreach**의 반복 변수는 **반복마다 새 변수**로 처리 → foreach 안의 람다는 각자 다른 캡처를 가짐.
- for에서는 여전히 위와 같이 동작하므로, 의도한 결과를 위해선:

```
for (int i = 0; i < 3; i++)
{
    int copy = i;
    actions.Add(() => Console.Write(copy + " "));
}
// 출력: 0 1 2
```

해설 93 — event vs delegate.

- Clicked?.Invoke(this, EventArgs.Empty) — 선언한 클래스 내부에서만 invoke 가능.
- 외부에서는 +=/--만 허용 → 캡슐화. 누가 함부로 Clicked = null로 모든 구독자를 날리거나 직접 호출할 수 없다.
- 이벤트는 boilerplate(add/remove accessor)를 자동 생성. 명시적 add/remove 로 lock/약한 참조 등 커스텀 가능.
- event 없이 raw delegate를 외부에 노출하면 사용자가 실수로 =로 덮어쓸 위험이 있다.

해설 94 — anonymous type.

- 컴파일러가 현재 어셈블리 한정 immutable class를 생성. 프로퍼티는 read-only.
- 두 anonymous type 인스턴스가 같은 프로퍼티 이름/타입/순서면 같은 타입을 공유.
- Equals/GetHashCode/ToString도 컴파일러가 생성(value-equality 의미).
- LINQ projection에서:

```
var q = from c in customers select new { c.Name, OrderCount = c.Orders.Count };
```

임시 결과 형태를 빠르게 표현. 단, 메서드 경계를 넘기기 어렵다(반환 타입을 적기가 곤란). 그런 경우 record 또는 명명된 DTO 사용.

26. 설계 및 종합 사고 — 해설

해설 95 — struct 적절성.

- **Point** → struct 적절. 작고(필드 2~3개) 불변에 가깝고 자주 단명. .NET BCL의 Point, Vector3 등이 모두 struct.
- **Color** → struct 적절. 4바이트(RGBA) 정도 작은 값.
- **Account** → class. 식별성(고유 ID), 가변 상태, 동일성/참조 의미가 중요.
- **Player** → class. 큰 상태, 식별성, 다른 시스템과의 관계 다수.

- **Person** → **class**. 식별성 있고 보통 상태가 가변, 멤버가 많음.

요약: “수학적인 작은 값”은 struct, “개체(entity)”는 class.

해설 96 — 합성 > 상속.

상속 설계 문제:

- 부모 변화가 자식 모두에 파급(fragile base class).
- 단일 상속 제약, 다중 분류(Duck은 Bird인데 동시에 Swimmer?, Flyer?) 어려움.
- “is-a”를 강요해 도메인이 어색해짐(OstrichBird는 IFlyable이 아닌데 부모가 Fly()를 갖는다면?).

합성 예:

```
interface IFlyBehavior { void Fly(); }
interface IQuackBehavior { void Quack(); }

class Duck
{
    private readonly IFlyBehavior _fly;
    private readonly IQuackBehavior _quack;
    public Duck(IFlyBehavior fly, IQuackBehavior quack) { _fly = fly; _quack = quack; }
    public void PerformFly() => _fly.Fly();
}

class FlyWithWings : IFlyBehavior { public void Fly() { /* ... */ } }
class NoFly : IFlyBehavior { public void Fly() { /* nothing */ } }
```

행위를 객체로 분리하고 주입해서 런타임에 조합. Strategy 패턴의 본질.

해설 97 — managed language.

- **CLR(Common Language Runtime)**: 메모리 관리(GC), 타입 안전, JIT 컴파일, 보안, 예외 등을 책임지는 런타임.
- **GC**: 메모리 자동 회수 → 누수와 dangling pointer 문제 대폭 감소.
- **메모리 안전성**: 배열 범위 검사, 타입 검사, 명시적 포인터 산술 금지(unsafe 컨텍스트 제외).
- C/C++ 같은 **unmanaged language**: 직접 메모리 할당/해제, 포인터 산술, 더 큰 성능과 더 큰 책임(use-after-free, buffer overflow 등).

C#도 unsafe/fixed/stackalloc/interop으로 unmanaged 영역과 만날 수 있다. 그러나 기본은 managed.

해설 98 — 종합 흐름.

예: HTTP API에서 상품 데이터를 받아 가공해 결과 보여주기.

1. **클래스/인터페이스**: 도메인 모델 Product(class), 데이터 소스 추상화 IProductRepository (Task<IEnumerable<Product>> GetAllAsync()) 정의.
2. **제네릭**: 공용 결과 컨테이너 Result<T>(성공/실패 + 데이터/오류) — 여러 API 응답에 재사용.
3. **async/await**: HttpClient.GetAsync 같은 I/O를 await로 호출 → UI/스레드 풀 효율 유지.
4. **예외 처리**: 네트워크 오류, 역직렬화 오류, 비즈니스 검증 실패 별로 try-catch로 처리하고 Result<T>로 변환.
5. **LINQ**: 받은 상품 목록을 Where(p => p.IsActive).GroupBy(p => p.Category).Select(...) 처럼 변환.

6. 결과: 호출 측은 비동기로 결과를 받아 UI에 바인딩.

```

public class ProductService
{
    private readonly IProductRepository _repo;
    public ProductService(IProductRepository repo) => _repo = repo;

    public async Task<Result<IReadOnlyList<CategoryAvg>>> GetCategoryAveragesAsync(
        CancellationToken token = default)
    {
        try
        {
            var products = await _repo.GetAllAsync(token);
            var result = products
                .Where(p => p.IsActive)
                .GroupBy(p => p.Category)
                .Select(g => new CategoryAvg(g.Key, g.Average(p => p.Price)))
                .ToList();
            return Result<IReadOnlyList<CategoryAvg>>.Ok(result);
        }
        catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound)
        {
            return Result<IReadOnlyList<CategoryAvg>>.Fail("데이터 소스를 찾을 수 없습니다.");
        }
        catch (OperationCanceledException)
        {
            throw; // 취소는 그대로 위로
        }
        catch (Exception ex)
        {
            return Result<IReadOnlyList<CategoryAvg>>.Fail($"알 수 없는 오류: {ex.Message}");
        }
    }
}

```

여기에서 클래스/인터페이스(IProductRepository), 제네릭(Result<T>), LINQ(Where/GroupBy/Select), async/await, 예외 처리(try-catch + when filter)가 모두 한 함수 안에 자연스럽게 어울려 등장한다.

해설 99 — SOLID.

S — Single Responsibility: 한 클래스는 한 가지 변경 이유만.

```

// 나쁨: 한 클래스가 저장 + 인쇄 + 메일 전송까지
class Report { void Save(){} void Print(){} void EmailTo(string to){} }

// 좋음
class Report { /* ... */ }
class ReportSaver { void Save(Report r){} }
class ReportPrinter { void Print(Report r){} }
class ReportMailer { Task EmailAsync(Report r, string to) => Task.CompletedTask; }

```

O — Open/Closed: 확장에는 열리고, 변경에는 닫혀야.

```

interface IDiscount { decimal Apply(decimal price); }
class NoDiscount : IDiscount { public decimal Apply(decimal p) => p; }
class PercentDiscount : IDiscount { /* ... */ public decimal Apply(decimal p) => p * 0.9m; }
// 새 할인 추가 시 기존 클래스 수정 없음.

```

L — Liskov Substitution: 자식은 부모의 계약을 깨지 않아야.

```

// 위반 예: 정사각형은 직사각형의 set 계약을 깬다.
class Rectangle { public virtual int Width {get;set;} public virtual int Height {get;set;} }
class Square : Rectangle {
    public override int Width { set { base.Width = base.Height = value; } get => base.Width; }
}
// 호출자가 W/H를 독립적으로 다룬다는 계약이 깨짐.

```

I — Interface Segregation: 사용자가 안 쓰는 메서드까지 떠안지 않게.

```
interface IPrintable { void Print(); }
interface IScannable { void Scan(); }
class OldPrinter : IPrintable { public void Print() {} }
class AllInOne : IPrintable, IScannable { public void Print() {} public void Scan() {} }
```

D — Dependency Inversion: 상위 모듈은 하위 모듈에 의존하지 말고, 둘 다 추상에 의존.

```
class OrderService
{
    private readonly IPaymentGateway _pay;
    public OrderService(IPaymentGateway pay) => _pay = pay;
}
```

구체 클래스(StripeGateway)는 DI 컨테이너가 주입.

해설 100 — C# vs .NET.

- **C#:** 언어(문법, 타입 시스템, 컴파일러).
- **.NET:** 플랫폼(런타임 + 클래스 라이브러리 + 도구).
- **CLR:** 런타임. JIT, GC, 보안, 스레드, 예외 등 실행 환경.
- **CTS(Common Type System):** 모든 .NET 언어가 공유하는 타입 시스템. `int = System.Int32` 같은 통일.
- **CLS(Common Language Specification):** .NET 언어 간 상호운용을 위한 최소 공통 규칙. CLS 호환 API는 어떤 .NET 언어에서도 호출 가능.
- **IL / JIT:** C# 컴파일러(csc)는 소스를 **IL(Intermediate Language)** + 메타데이터를 가진 어셈블리(.dll/.exe)로 변환. 실행 시 **JIT**가 IL을 호스트 CPU의 기계어로 컴파일. **AOT**(ReadyToRun, NativeAOT)로 시작 시간 단축 가능.

플랫폼 변천

- **.NET Framework:** Windows 전용, 4.x가 마지막.
- **.NET Core:** 크로스 플랫폼(Win/Linux/macOS), 빠른 릴리스.
- **.NET 5+:** Core를 베이스로 Framework/Mono/Xamarin을 통합한 단일 .NET. 매년 한 번 메이저 릴리스(작은 LTS).

다른 .NET 언어와 상호운용

- F#(함수형), VB.NET(절차/객체)도 같은 CLR + CTS 위에서 동작 → 같은 어셈블리 참조, 같은 타입 사용 가능.
- C#로 만든 라이브러리(예: ASP.NET Core)는 F#에서 그대로 호출 가능. 반대도 성립.

이 통합 덕분에 언어는 도구로 선택하고, 플랫폼은 **.NET** 하나로 통일하는 것이 현재 .NET 생태계의 핵심이다.

끝맺음

100문제를 모두 풀어본 학생이라면 C#의 문법을 넘어, 왜 그렇게 설계되었는가에 대한 감각을 가지게 되었을 것이다. 다음 학습 방향:

- Unity의 MonoBehaviour / 코루틴 / 이벤트 시스템에 본 개념을 적용해 보기
- ASP.NET Core, EF Core 등 실제 응용 프레임워크 학습

- `Span<T>`, `record`, 패턴 매칭 등 최신 C# 기능을 활용한 리팩토링 연습